

### Cerințe obligatorii

1. Pattern-urile implementate trebuie să respecte definiția din GoF discutată în cadrul cursurilor și laboratoarelor. Nu sunt acceptate variații sau implementări incomplete.
2. Pattern-ul trebuie implementat în totalitate corect pentru a fi luat în calcul.
3. Soluția nu conține erori de compilare.
4. Pattern-urile pot fi tratate distinct sau pot fi implementate pe același set de clase.
5. Implementările care nu au legătura funcțională cu cerințele din subiect NU vor fi luate în calcul (preluare unui exemplu din alte surse nu va fi punctată).
6. NU este permisă modificarea claselor primite.
7. Soluțiile vor fi verificate încrucișat folosind MOSS. Nu este permisă partajarea de cod între studenți. Soluțiile care au un grad de similitudine mai mare de 30% vor fi anulate.

### Cerințe Clean Code obligatorii (soluția este depunctată cu câte 2 puncte pentru fiecare cerință ce nu este respectată) - maxim se pot pierde 8 puncte

1. Pentru denumirea claselor, funcțiilor, testelor unitare, atributelor și a variabilelor se respecta convenția de nume de tip Java Mix CamelCase;
2. Pattern-urile și clasa ce conține metoda main() sunt definite în pachete distincte ce au forma *cts.nume.prenume.gNrGrupa.denumire\_pattern*, *cts.nume.prenume.gNrGrupa.main* (studenții din anul suplimentar trec "as" în loc de gNrGrupa);
3. Clasele și metodele sunt implementate respectând principiile KISS, DRY și SOLID (atenție la DIP);
4. Denumirile de clase, metode, variabile, precum și mesajele afișate la consola trebuie să aibă legătura cu subiectul primit (nu sunt acceptate denumiri generice). Funcțional, metodele vor afișa mesaje la consola care să simuleze acțiunea cerută sau vor implementa prelucrări simple.

### Se dezvoltă o aplicație software destinată unui magazin.

- 4p.** În cadrul unui magazin se dorește implementarea unui modul ce facilitează procesul de generare bon. Astfel, este restricționată deținerea în același timp a mai multor comenzi în desfășurare, aplicația permițând operații de modificare doar pe unica comandă în curs de desfășurare. Clasa trebuie să implementeze interfața AbstractCasaMarcat și să permită operațiile de consultare stare curentă comandă, adăugare produs nou, eliminare produs, închidere și deschidere comandă nouă. Se cunoaște faptul că există mai mulți angajați (derivă din clasa AbstractAngajat) care pot interacționa cu modulul implementat iar o comandă odată deschisă este gestionată doar de angajatul care a adăugat-o.
- 2p.** Să se testeze soluția prin simularea procesului de deschidere comandă de către un angajat și adăugarea de modificări pe comanda curentă. Să se demonstreze că o comandă nu poate fi modificată decât de către angajatul în cauză.
- 2p.** Se dorește extinderea modului propus prin posibilitatea de a permite existența a două case de marcat. Restricțiile prezentate anterior se păstrează și în curenta implementare.
- 1p.** Să se testeze soluția prin simularea procesului de deschidere comandă de către doi angajați folosind două fire de execuție diferite.

```
public interface AbstractAngajat {  
    String getNume();
```

```
}
```

```
public interface AbstractCasaMarcat {  
    void deschideComanda(AbstractAngajat angajat);  
    void inchideComanda(AbstractAngajat angajat);  
    void adaugaProduce(String denumireProduce);  
    void showInfoComanda();  
}
```